ADVANCED RAG SERIES | PART 1 OF 5

Query Transformation, Routing and Construction

Pre-retrieval optimization for
Advanced RAG systems

Covers: query rewriting, history condensation, multi-query and RAG-Fusion, HyDE, step-back prompting, decomposition, routing, self-query, Text-to-SQL.

A practitioner's guide: concepts, trade-offs, code and checklists

Contents

1. Why the Query Is the Weakest Link 3

1.1 The query-document gap . . . 3
1.2 A taxonomy of query failures . . . 3
1.3 Where pre-retrieval fits in the pipeline . . . 3

2. Query Rewriting 4

2.1 What a good rewrite looks like . . . 4
2.2 Implementation . . . 4
2.2.1 Failure modes . . . 5

3. History-Aware Query Condensation 5

4. Multi-Query Retrieval and RAG-Fusion 6

4.1 Generating the variants . . . 6
4.2 Merging results with Reciprocal Rank Fusion . . . 6

5. HyDE: Hypothetical Document Embeddings 7

5.1 Why it works . . . 7
5.2 Implementation . . . 7
5.3 When HyDE hurts . . . 8

6. Step-Back Prompting 8

7. Decomposition and Multi-Hop Reasoning 9

7.1 Independent sub-questions . . . 9
7.2 Sequential (least-to-most) decomposition . . . 9
7.3 Interleaving retrieval with reasoning (IRCoT) . . . 9

8. Query Routing 9

8.1 Logical routing with an LLM . . . 10
8.2 Semantic routing . . . 10
8.3 Choosing a routing strategy . . . 10

9. Query Construction: From Language to Structured Queries 11

9.1 Self-querying retrieval . . . 11
9.2 Text-to-SQL and Text-to-Cypher . . . 11

10. Putting It Together 12

10.1 A decision guide . . . 12
10.2 A conditional pipeline . . . 12
10.3 Evaluating query transformations . . . 13
10.4 Checklist . . . 13
10.5 Summary and further reading . . . 13

1. Why the Query Is the Weakest Link

A basic retrieval-augmented generation (RAG) system has a deceptively simple shape: embed the user's question, fetch the nearest chunks from a vector store, paste them into a prompt, and let the language model answer. This works surprisingly well on demos, where the questions are written by the same person who wrote the documents. It breaks down in production, where questions come from real users who are vague, terse, conversational, multi-part, or who use vocabulary that the corpus never uses. The retrieval stage can only be as good as the query it is handed, and the raw user message is frequently a poor query.

Pre-retrieval optimization is the family of techniques that sits between the user's message and the index. Instead of treating the question as sacred input, an advanced RAG system treats it as a **draft** that can be rewritten, expanded, split, translated into a structured filter, or routed to a different data source altogether. This document walks through each of those techniques, shows when each one helps, what it costs in latency and tokens, and how to implement it with a modern LLM stack.

1.1 The query-document gap

Dense retrieval compares a query embedding to chunk embeddings. Queries and answers, however, are different kinds of text. A query is short, interrogative and often underspecified ("why is my pump noisy?"). A relevant passage is long, declarative and technical ("Cavitation occurs when local static pressure falls below the vapour pressure of the fluid..."). Embedding models are trained to bridge this gap, but the bridge is imperfect, particularly in specialised domains and for out-of-distribution phrasing. Most pre-retrieval techniques are, at heart, different strategies for narrowing this gap: make the query look more like the documents, make it more precise, or make it more complete.

1.2 A taxonomy of query failures

Before choosing a technique, it helps to name the failure you are actually seeing. The table below maps common failure patterns to the technique that usually addresses them. Diagnosing first prevents the common mistake of stacking every transformation on every query, which inflates cost and can even reduce quality.

Failure pattern	Example	Typical remedy
Vocabulary mismatch	User says "car won't start"; manual says "ignition failure"	Query rewriting, multi-query, HyDE
Under-specified or terse	"pricing?"	Rewriting with context, clarification step
Conversation dependence	"And what about the second one?"	History-aware condensation
Too specific, answer is general	"Why did Python 3.12 change X in module Y?"	Step-back prompting
Multi-part or multi-hop	"Compare the warranty of product A with the return policy of B"	Decomposition, IRCoT
Needs structured filtering	"Papers on RAG published after 2023 by Google"	Self-query / metadata filters
Wrong data source	"How many orders last week?"	Routing to SQL; Text-to-SQL

Table 1. Query failure patterns and the technique that usually addresses them.

1.3 Where pre-retrieval fits in the pipeline

The diagram below shows an advanced pipeline. The first three boxes are the focus of this document. Notice that they are **optional and composable**: a production system typically enables only the stages that its own failure analysis justifies, and often applies them conditionally based on a cheap classifier.

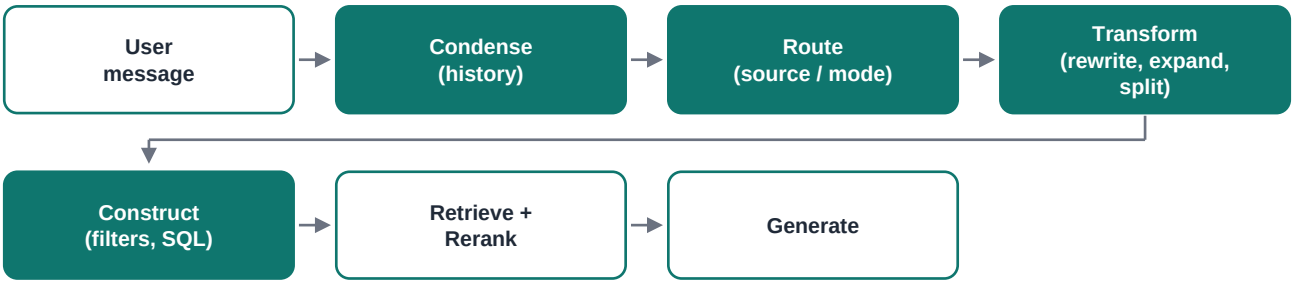


Figure 1. A pre-retrieval stack. Shaded boxes are covered in this document.

> **KEY IDEA** Every transformation is an extra LLM call or an extra retrieval. Treat query transformation as a budgeted resource: measure the retrieval gain per added millisecond and per added token before adopting it.

2. Query Rewriting

Query rewriting asks a language model to restate the user's question as a better search query. The pattern was formalised as **Rewrite-Retrieve-Read** (Ma et al., 2023), which observed that the retrieval query need not be the question itself and can be optimised separately from the reader model. In practice the rewriter removes filler words, resolves abbreviations, adds domain terminology, and strips conversational noise.

2.1 What a good rewrite looks like

Consider the user message: "hey so my laptop keeps dying really fast after the update, any idea what's up??". The embedding of this sentence is dominated by conversational tokens. A rewriter produces something like: "laptop battery drains quickly after operating system update: causes and fixes". The second string contains the entities and the intent, and nothing else. Dense retrievers respond very differently to the two.

2.2 Implementation

The simplest rewriter is a prompt, a model and an output parser. Keep temperature at zero so that the same query always rewrites the same way, which makes your evaluation reproducible. Pass the corpus domain in the system prompt so that the model uses the right vocabulary.

```
from langchain_groq import ChatGroq
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

llm = ChatGroq(model="llama-3.3-70b-versatile", temperature=0)

rewrite_prompt = ChatPromptTemplate.from_messages([
    ("system",
     "You rewrite user questions into concise search queries for a technical "
     "knowledge base about {domain}. Keep every named entity, number and "
     "constraint. Remove greetings and filler. Expand abbreviations. "
     "Return ONLY the rewritten query."),
    ("human", "{question}"),
])

rewriter = rewrite_prompt | llm | StrOutputParser()

q = rewriter.invoke({
    "domain": "laptop hardware and operating systems",
    "question": "hey so my laptop keeps dying really fast after the update??",
})
print(q)
```

Listing 1. A minimal query rewriter using LangChain expression language.

Two details matter more than they appear. First, instruct the model to **preserve** named entities, version numbers and negations: a rewriter that drops "not" or "2023" silently destroys the question. Second, retrieve with **both** the original and the rewritten query and merge the results (see Section 4 on fusion). The original query is a safety net against the occasional bad rewrite.

2.2.1 Failure modes

- **Over-specification.** The model hallucinates detail that the user never mentioned, steering retrieval toward the wrong documents.
- **Drift.** The rewrite answers a slightly different question. This is common with small models; prefer a capable model for rewriting, or fine-tune a small one.
- **Latency.** An additional sequential LLM call adds hundreds of milliseconds. Use a fast model, and skip rewriting for queries that are already well-formed (for example, long queries with several domain terms).
- **Language mismatch.** If the corpus is English and users write in Hindi, Urdu or code-mixed text, the rewriter can also translate, but you must test that named entities survive transliteration.

3. History-Aware Query Condensation

Chat interfaces create a problem that single-shot search does not have: **the current message is often meaningless on its own**. "What about the second one?" or "Does it support streaming?" cannot be embedded usefully without the preceding turns. The standard fix is to condense the chat history and the latest message into a standalone question before retrieval.

A standalone question must be self-contained: it should be understandable by someone who never saw the conversation. Notice that this is a stronger requirement than "include the topic". Pronouns, ellipsis, and relative references ("the previous table", "that library") must all be resolved to explicit entities.

```
from langchain_core.messages import HumanMessage, AIMessage
from langchain_core.prompts import MessagesPlaceholder

condense_prompt = ChatPromptTemplate.from_messages([
    ("system",
     "Given the chat history and the latest user message, write a standalone "
     "question that can be understood without the history. Resolve pronouns "
     "and references. If the message is already standalone, return it unchanged. "
     "Return ONLY the question."),
    MessagesPlaceholder("history"),
    ("human", "{question}"),
])
condenser = condense_prompt | llm | StrOutputParser()

history = [
    HumanMessage("Tell me about FAISS."),
    AIMessage("FAISS is a library for efficient similarity search over dense vectors."),
]
print(condenser.invoke({"history": history, "question": "Does it support GPUs?"}))
# -> "Does the FAISS library support GPU acceleration?"
```

Listing 2. History-aware condensation. The output is used for retrieval, not shown to the user.

Condense only for retrieval; **do not** replace the user-visible message with the condensed one. The final answer-generation prompt should still receive the real conversation so that tone and continuity are preserved. A common optimisation is to condense only when the history is non-empty and the latest message contains a pronoun, a demonstrative, or is shorter than a few words, avoiding an unnecessary call on the first turn.

PITFALL Condensing the entire history every turn can accumulate errors: the condensed question from turn 4 may already contain a misreading from turn 2. Cap the history window (for example, the last 4 to 6 messages) and always give the condenser the raw messages, not previous condensed questions.

4. Multi-Query Retrieval and RAG-Fusion

A single query embedding is a single point in vector space, and the nearest neighbours of one point can miss relevant documents that sit in a slightly different direction. **Multi-query retrieval** generates several paraphrases or perspectives of the question, runs retrieval for each, and unions the results. The intuition is the same as asking a librarian the same question in several ways: each phrasing surfaces a different part of the shelf.

4.1 Generating the variants

Ask the model for three to five alternatives that differ meaningfully, not trivially. Useful axes of variation are terminology (synonyms and technical terms), granularity (broader and narrower), and viewpoint (the question as a beginner might ask it versus as an expert would). Request one variant per line and parse by splitting on newlines.

```
multi_prompt = ChatPromptTemplate.from_messages([
    ("system",
     "Generate {n} different search queries that would help answer the user's "
     "question. Vary terminology and level of detail. One query per line, "
     "no numbering, no commentary."),
    ("human", "{question}"),
])
multi_gen = multi_prompt | llm | StrOutputParser()

def expand(question: str, n: int = 4) -> list[str]:
    raw = multi_gen.invoke({"question": question, "n": n})
    variants = [l.strip() for l in raw.split("\n") if l.strip()]
    return [question] + variants[:n]      # always keep the original
```

Listing 3. Multi-query expansion that always keeps the original query.

4.2 Merging results with Reciprocal Rank Fusion

After retrieving for every variant you hold several ranked lists. Simply concatenating them loses ranking information; averaging similarity scores is fragile because scores from different queries are not calibrated against each other. **Reciprocal Rank Fusion (RRF)** (Cormack et al., 2009) sidesteps the calibration problem by using ranks only. Each document receives a score of 1 / (k + rank) from every list in which it appears, and scores are summed. The constant k (commonly 60) damps the influence of the very top ranks so that one list cannot dominate. Documents that appear in several lists rise to the top, which is exactly the consensus signal you want. The combination of multi-query generation and RRF is often called **RAG-Fusion**.

```
from collections import defaultdict

def rrf(ranked_lists, k=60, top_n=8):
    """ranked_lists: list of lists of (doc_id, doc) in best-first order."""
    scores, docs = defaultdict(float), {}
    for lst in ranked_lists:
        for rank, (doc_id, doc) in enumerate(lst, start=1):
            scores[doc_id] += 1.0 / (k + rank)
            docs[doc_id] = doc
    best = sorted(scores, key=scores.get, reverse=True)[:top_n]
    return [(d, docs[d], scores[d]) for d in best]

# usage
variants = expand("How do I reduce hallucinations in RAG?")
lists = []
for v in variants:
    hits = retriever.invoke(v)                       # any LangChain retriever
    lists.append([(h.metadata["chunk_id"], h) for h in hits])
fused = rrf(lists)
```

Listing 4. Reciprocal Rank Fusion in about ten lines of plain Python.

RRF has two attractive properties for production. It needs no training data, and it is robust to retrievers with different score scales, which is why it reappears in Part 3 of this series when we fuse keyword and vector search. The cost is linear in the number of variants: four variants means four retrievals. Because retrievals are independent, run them concurrently so that latency grows with the slowest call rather than the sum.

> **TIP** Cap the number of variants at three to five. Quality gains flatten quickly while cost keeps growing, and extra variants increase the chance that one of them drifts off-topic and injects noise.

5. HyDE: Hypothetical Document Embeddings

HyDE (Gao et al., 2022) attacks the query-document gap from the opposite direction. Instead of making the query look better, it makes the query look like a **document**. The model is asked to write a plausible answer to the question, a hypothetical passage, and the embedding of that passage is used for retrieval. The passage may contain factual errors, and that is acceptable, because the retriever uses it only to locate real documents with similar content and wording. The real documents then ground the final answer.

5.1 Why it works

An answer passage shares vocabulary, phrasing and length with corpus documents, so its embedding falls close to relevant chunks. In the original paper this made unsupervised dense retrieval competitive without any relevance labels. The technique shines on zero-shot, domain-specific corpora where no fine-tuned embedding model exists, and on short, abstract questions whose embeddings are far from long technical passages.

5.2 Implementation

```
hyde_prompt = ChatPromptTemplate.from_messages([
    ("system",
     "Write a short passage (3-5 sentences) in the style of a technical "
     "reference document that answers the question. Be specific and use "
     "domain terminology. It is fine if details are uncertain."),
    ("human", "{question}"),
])
hyde_gen = hyde_prompt | llm | StrOutputParser()

def hyde_retrieve(question: str, vectorstore, k: int = 6):
    fake_doc = hyde_gen.invoke({"question": question})
    # embed the hypothetical passage, NOT the question
    return vectorstore.similarity_search(fake_doc, k=k)
```

Listing 5. HyDE retrieval. The generated passage is used only as a search key.

5.3 When HyDE hurts

- **Obscure or recent topics.** If the model knows nothing about the subject, the hypothetical passage is confidently wrong in a way that points to the wrong region of the index. HyDE leans on the model's prior knowledge.
- **Keyword-critical queries.** Product codes, error numbers and exact names are best matched lexically. A fabricated passage may substitute different identifiers.
- **Latency-sensitive paths.** Generating a paragraph is slower than generating a short rewrite. Use a small fast model, cap tokens, or enable HyDE only after a first-pass retrieval returns low-confidence scores.

A pragmatic production pattern is to combine HyDE with the original query: retrieve with both embeddings and fuse with RRF. The original query anchors precision, and the hypothetical passage adds recall.

6. Step-Back Prompting

Step-back prompting (Zheng et al., 2023) addresses the opposite problem from vague queries: questions that are **too specific**. When a user asks about a narrow detail, the documents that contain the governing principle may not mention the detail at all. By first asking a more abstract question, the system retrieves background knowledge that makes the specific answer derivable.

For example, the question "What happens to the pressure of an ideal gas if its temperature is doubled and its volume is halved?" steps back to "What are the relationships between pressure, volume and temperature for an ideal gas?". The retrieval for the second question returns the ideal gas law, from which the model can compute the answer. The technique uses **two retrievals**: one for the step-back question and one for the original, and the generation prompt receives both sets of context.

```
stepback_prompt = ChatPromptTemplate.from_messages([
    ("system",
     "Rewrite the question as a more general, abstract question about the "
     "underlying concept or principle. Return ONLY the general question."),
    ("human", "{question}"),
])
stepback = stepback_prompt | llm | StrOutputParser()

def stepback_context(question, retriever):
    general_q = stepback.invoke({"question": question})
    specific = retriever.invoke(question)
    general = retriever.invoke(general_q)
    return general + specific        # principles first, details second
```

Listing 6. Step-back retrieval with two parallel contexts.

Step-back works best for science, engineering, law and any domain where specific cases are instances of general rules. It is less useful for lookup questions ("What is the refund window?"), where the specific passage is the answer. Few-shot examples in the prompt substantially improve the quality of the abstraction.

7. Decomposition and Multi-Hop Reasoning

Some questions cannot be answered by any single chunk. "Who is older, the founder of the company that makes the A100 or the founder of the company that makes the MI300?" requires identifying two companies, then two founders, then two birth dates, then a comparison. A single retrieval on the whole sentence will find passages about neither founder. **Decomposition** breaks the question into sub-questions that are individually retrievable.

7.1 Independent sub-questions

When sub-questions do not depend on each other, ask the model for a list, retrieve for each in parallel, and answer them separately before a final synthesis step. This is the simplest form and the easiest to parallelise.

7.2 Sequential (least-to-most) decomposition

When later steps depend on earlier answers, as in the founder example, the sub-questions must be answered in order, feeding each answer into the next question. This follows the **least-to-most** prompting idea: solve simple sub-problems first and reuse their solutions. The loop retrieves, answers, appends the answer to a scratchpad, and asks the model for the next sub-question until it can answer the original.

7.3 Interleaving retrieval with reasoning (IRCoT)

IRCoT (Trivedi et al., 2022) interleaves retrieval with chain-of-thought. After each reasoning sentence, the model's latest thought becomes the next retrieval query, so the evidence collected evolves with the reasoning. It improved multi-hop question answering because later retrievals can use entities that were unknown at the start. This is the conceptual ancestor of agentic RAG, covered in Part 4.

```
from pydantic import BaseModel, Field

class SubQuestions(BaseModel):
    sub_questions: list[str] = Field(
        description="Self-contained sub-questions, in dependency order, max 4")

planner = llm.with_structured_output(SubQuestions)

def answer_multi_hop(question, retriever, answer_chain):
    plan = planner.invoke(f"Decompose into sub-questions: {question}")
    notes = []
    for sq in plan.sub_questions:
        # substitute earlier answers into later questions if needed
        ctx = retriever.invoke(sq)
        ans = answer_chain.invoke({"question": sq, "context": ctx, "notes": notes})
        notes.append(f"Q: {sq}\nA: {ans}")
    return answer_chain.invoke({"question": question, "context": [], "notes": notes})
```

Listing 7. Plan-then-execute decomposition with structured output.

> **PITFALL** Decomposition multiplies cost and compounds errors: if the first sub-answer is wrong, every later step builds on it. Always set a hard limit on the number of sub-questions, log every intermediate answer, and let the final step state when evidence for a sub-question was missing instead of guessing.

8. Query Routing

Real applications rarely have a single index. A company assistant may need to consult a product-documentation vector store, an HR policy collection, a SQL warehouse of orders, a graph of org relationships, and a web search tool. **Routing** decides, per query, where to send it, and sometimes whether to retrieve at all.

8.1 Logical routing with an LLM

In logical routing, the model is shown a description of each data source and chooses one (or several) by name. Using structured output keeps the decision machine-readable. Write source descriptions the way you would brief a new colleague: what the source contains, what it does not contain, and an example question.

```
from typing import Literal

class Route(BaseModel):
    source: Literal["product_docs", "hr_policies", "sql_orders", "web", "no_retrieval"]
    reason: str = Field(description="One short sentence")

router = llm.with_structured_output(Route)

SYSTEM = """Route the question to the best source.
product_docs : features, setup, troubleshooting of our products
hr_policies  : leave, benefits, code of conduct, internal rules
sql_orders   : counts, totals, trends about orders, customers, revenue
web          : news and facts that change over time
no_retrieval : greetings, chit-chat, simple arithmetic"""

decision = router.invoke([("system", SYSTEM), ("human", "Total revenue in September?")])
print(decision.source)      # sql_orders
```

Listing 8. LLM router with a closed set of options and a short justification.

8.2 Semantic routing

Semantic routing embeds the query and compares it with example utterances for each route, picking the nearest. It needs no generation call, so it is fast and cheap, typically a few milliseconds once embeddings are cached. It is ideal as a first gate: handle the obvious cases semantically and fall back to the LLM router only when the best similarity is below a threshold.

8.3 Choosing a routing strategy

Strategy	Latency	Accuracy on edge cases	Best for
Rules / regex	Microseconds	Brittle	Hard constraints (e.g., contains an order ID)
Semantic (embedding)	Milliseconds	Good with 10-20 examples per route	High-volume first-pass routing
LLM router	Hundreds of ms	Best, can reason about ambiguity	Few routes with subtle boundaries
Trained classifier	Milliseconds	Best with labelled traffic	Mature systems with logged queries

Table 2. Routing strategies compared.

Adaptive RAG (Jeong et al., 2024) extends the idea: a small classifier estimates the **complexity** of the query and chooses between no retrieval, single-step retrieval, and an iterative multi-step pipeline. This spends computation only where it pays off, and we return to it in Part 4.

9. Query Construction: From Language to Structured Queries

Not every question is best served by semantic similarity. "Papers about retrieval published after 2023 by Google" has a semantic component (retrieval) and three structured constraints (a date, an organisation, a document type). Embedding the whole sentence blurs the constraints into the semantics and returns approximate matches. **Query construction** translates natural language into the query language of the target store.

9.1 Self-querying retrieval

A self-query retriever asks the model to split the user's sentence into a semantic query string and a set of metadata filters expressed against a known schema. The vector store then applies the filters **exactly** and ranks the remaining chunks by similarity. This requires that metadata (author, year, category, product, access level) was captured at indexing time, one of the reasons Part 2 of this series stresses metadata enrichment.

```
class SearchSpec(BaseModel):
    query: str = Field(description="Semantic part of the question")
    year_min: int | None = None
    organisation: str | None = None
    doc_type: Literal["paper", "blog", "docs"] | None = None

spec = llm.with_structured_output(SearchSpec).invoke(
    "Papers about retrieval published after 2023 by Google")

flt = {}
if spec.year_min:      flt["year"] = {"$gte": spec.year_min}
if spec.organisation:  flt["org"] = spec.organisation
if spec.doc_type:      flt["type"] = spec.doc_type

hits = vectorstore.similarity_search(spec.query, k=6, filter=flt)
```

Listing 9. Self-query: LLM produces a structured spec, the application builds the filter.

Note that the LLM emits **values**, not raw filter syntax. Constructing the actual filter in application code is safer: you validate fields against the schema, reject unknown operators, and avoid injecting model output directly into a database call. Filter syntax also differs between stores, and keeping it in code isolates that difference.

9.2 Text-to-SQL and Text-to-Cypher

For questions over tabular or graph data, retrieval means executing a query. Text-to-SQL prompts the model with the database schema (table names, column names and types, a few sample rows, and key relationships) and asks for a SQL statement. A robust pipeline adds several stages around the generation step.

1. **Schema selection.** For large schemas, retrieve only the relevant tables with embeddings rather than passing the entire schema.
2. **Generation.** Produce a SQL statement for the specific dialect (MySQL, PostgreSQL, SQLite).
3. **Validation.** Parse the statement, allow only read-only SELECT queries, restrict tables, and add a row limit.
4. **Execution and repair.** Run the query; on an error, feed the message back to the model for a bounded number of repair attempts.
5. **Answer synthesis.** Convert the result rows to a natural-language answer, quoting the numbers exactly.

Text-to-Cypher follows the same recipe for graph databases such as Neo4j, using the graph schema in place of table definitions. Both techniques appear again in Part 4 in the context of graph-based and agentic RAG, and Part 5 covers how to evaluate their correctness.

> **PITFALL** Never give a Text-to-SQL pipeline write access. Run it with a read-only database user, enforce a statement timeout and a row limit, and log every generated query. Prompt-injected content in retrieved text can otherwise turn into destructive statements.

10. Putting It Together

10.1 A decision guide

There is no universal best transformation. The table summarises the cost and benefit profile so that you can pick based on observed failures rather than fashion.

Technique	Extra LLM calls	Extra retrievals	Strongest when	Watch out for
Rewrite	1	0	Noisy, conversational queries	Drift, dropped negations
Condense history	1	0	Multi-turn chat	Error accumulation
Multi-query + RRF	1	3-5	Recall is the bottleneck	Cost, off-topic variants
HyDE	1	1	Zero-shot domains, short queries	Unknown topics, exact IDs
Step-back	1	2	Principle-based questions	Lookup questions
Decomposition	2 or more	2 or more	Multi-hop, comparisons	Compounding errors
Routing	0 or 1	0	Multiple sources	Mis-routing; need fallback
Self-query	1	1	Metadata-rich corpora	Schema drift, bad filters

Table 3. Cost and risk profile of pre-retrieval techniques.

10.2 A conditional pipeline

The most effective systems apply transformations **conditionally**. A cheap gate classifies each query, and only the queries that need help receive extra treatment. A reasonable first version looks like this:

1. If there is chat history and the message is not standalone, condense it.
2. Run the router. If the answer is "no retrieval", answer directly; if it is a structured source, switch to Text-to-SQL or Text-to-Cypher.
3. For document sources, run a first retrieval with the (condensed) query and inspect the top similarity or reranker score.
4. If confidence is high, continue. If it is low, escalate: multi-query with fusion, then HyDE, then decomposition if the query contains several clauses.
5. Always log which stages fired, so that evaluation can attribute gains and losses to a specific stage.

10.3 Evaluating query transformations

Because transformations change what is retrieved, evaluate them at the **retrieval** level first, using a labelled set of questions with known relevant chunks. Track recall at k, mean reciprocal rank, and the fraction of queries where the correct chunk entered the top k only because of the transformation (wins) versus those where it fell out (regressions). A technique with positive average gain but frequent regressions on a particular query class should be gated off for that class. Part 5 covers metrics and datasets in detail.

10.4 Checklist

- Collect 50 to 200 real user queries and label the failures before building anything.
- Start with history condensation and a single rewrite; measure.
- Add multi-query with RRF only if recall at k is the problem.
- Add HyDE for abstract questions on specialised corpora; keep the original query in the mix.
- Add routing as soon as you have more than one data source; keep a safe default route.
- Validate and sandbox every generated filter or SQL statement.
- Cache transformed queries (keyed by normalised text) to cut cost and latency on repeated questions.
- Log the original, transformed and final queries together with retrieval scores.

10.5 Summary and further reading

Pre-retrieval optimization improves RAG by fixing the input to the retriever rather than the retriever itself. Rewriting and condensation clean the query; multi-query and HyDE enlarge the search net; step-back and decomposition reshape the question for general and multi-part needs; routing and query construction send each question to the source and query language that can actually answer it. The best systems combine a few of these, apply them conditionally, and measure each one.

Key references: Ma et al., *Query Rewriting for Retrieval-Augmented Large Language Models* (2023); Gao et al., *Precise Zero-Shot Dense Retrieval without Relevance Labels* (HyDE, 2022); Zheng et al., *Take a Step Back: Evoking Reasoning via Abstraction in Large Language Models* (2023); Trivedi et al., *Interleaving Retrieval with Chain-of-Thought Reasoning* (IRCoT, 2022); Cormack et al., *Reciprocal Rank Fusion outperforms Condorcet and individual rank learning methods* (2009); Jeong et al., *Adaptive-RAG* (2024).

Next in the series: Part 2, *Advanced Indexing, Chunking and Embeddings*, which explains how to build the index that these queries run against.